

Optimisation BackEnd avec Java :

ETL & API REST autour de la data d'Open Food Facts

Description : 1 ETL en Java, c'est possible ?

Vous connaissez peut-être l'application **Yuka**, disponible sur nos smartphones.

Yuka fournit des informations nutritionnelles sur pratiquement tous les produits alimentaires commercialisés en France.

Nous allons donc nous inspirer de cette application à succès qui s'appuie sur une base de données publique nommée **Open Food Facts**. En effet, cette base contient des milliers de produits alimentaires avec leurs propriétés nutritionnelles de qualités de A (excellent) à F (mauvais).

L'objectif de ce projet est de construire un backend Java optimisé qui ingère, nettoie, stocke et expose ces données via une API REST moderne.

La base de données **Open Food Facts** est une base de données **mondiale**, qu'on peut télécharger sous la forme d'un fichier **CSV**. Le fichier que je vais vous demander de traiter dans le cadre de ce projet est le même que celui sur lequel s'est basé Yuka dans ses premières versions. Il ne concerne que **les produits alimentaires fabriqués en France**.

Dans ce TP vous allez donc créer un ETL au travers d'une application Spring Boot/Batch avec Data JPA qui extrait, traite et met en base le contenu de ce fichier.

Vous utiliserez des techniques d'optimisation telles que le multithreading avec Virtual Threads (Project Loom) et la mise en cache pour accélérer les traitements de votre ETL.

Contenu du fichier

Le fichier open food facts, au format CSV, comporte 13 432 références de produits avec 30 informations associées par produit. Dans ce fichier le caractère séparateur est le |. Si on découpe selon ce caractère séparateur, on obtient le tableau d'éléments suivant :

- Index 0 : catégorie du produit
- Index 1 : marque du produit
- Index 2 : nom du produit
- Index 3 : score nutritionnel : A (excellent) à F (mauvais)
- Index 4 : liste des ingrédients séparés la plupart du temps par des virgules, mais pas toujours.
- Index 5 : énergie pour 100g (en joules)
- Index 6 : quantité de graisse pour 100g
- etc.
- Index 28 : liste des allergènes séparés la plupart du temps par des virgules
- Index 29 : liste des additifs séparés la plupart du temps par des virgules

NB : Des nettoyages sont requis (suppression des caractères parasites, parenthèses, pourcentages, ...)

Objectifs

Le but est de **concevoir** et **développer** une application Spring Boot / Data / JPA / Rest pour mettre en base toutes ces données afin de les exposer à travers l'API Rest.

Les règles de gestion à respecter :

- o Une catégorie doit être unique en base de données
- o Une marque doit être unique en base de données
- o Un ingrédient doit être unique en base de données.
- o Un allergène doit être unique en base de données
- o Un additif doit être unique en base de données
- o Cas particulier du traitement des ingrédients :
 - Dans le fichier, les ingrédients sont séparés par un séparateur. Exemple : Sucre, farine, banane
 - Le séparateur le plus courant est la virgule mais il est possible que d'autres séparateurs aient été utilisés comme le tiret. En effet comme il s'agit d'une base mondiale, certains utilisateurs ne respectent pas toujours les consignes. Attention donc à prévoir plusieurs cas de figures.
 - De manière à éviter les ingrédients du type Sucre* ou _Sucre_ par exemple, il faudra prévoir une suppression des caractères parasites.
 - Certains ingrédients possèdent une description entre parenthèses : il est demandé de supprimer toutes les informations présentes entre parenthèses.
 - Certains ingrédients possèdent également des %. De manière à éviter d'avoir des ingrédients du type Sucre 50%, il faudra veiller à supprimer tous les pourcentages.

Cas d'exemples pour les traitements des ingrédients, des allergènes et des additifs :

Cas 1 – les caractères spéciaux :

Avant traitement : Sucre*, farine, _Maïs_

Après traitement : Sucre, farine, Maïs

Cas 2 – les pourcentages :

Avant traitement : Sucre 15%, farine 50%, Maïs 35%

Après traitement : Sucre, farine, Maïs

Cas 3 – les parenthèses :

Avant traitement : Sucre, banane, Pâte (Farine 50%, Sucre 20%, Œufs 30%)

Après traitement : Sucre, banane, Pâte

OBJECTIF N°1 : Conception

- Réaliser un diagramme de classes métier
- Réaliser un modèle physique de données (ou MLD)
- Prévoir 1/2 journée pour ce travail sinon vous n'aurez pas le temps de réaliser le développement.

- Placer le dossier de conception dans un répertoire appelé **conception** et situé à la racine de votre projet.

OBJECTIF N°2 : Développement

- Mettre en en place les entités JPA avec les annotations
- Développer de manière professionnelle en suivant les bonnes pratiques de codage : javadoc, pas de code redondant, etc.
- Implémenter les couches « service » et « DAO » (Data Access Object - une DAO par entité métier pour les accès à la base de données. <https://www.baeldung.com/java-dao-pattern>)

OBJECTIF N°3 : Optimisation des performances

- Implémenter un système de cache (ex : Ehcache ou Spring Cache)
- Utiliser les Virtual Threads (Project Loom) pour paralléliser le traitement des lignes du fichier
- Analyser les performances (temps total, threads CPU, mémoire)
- Tenter de battre le record de traitement de **3min et 45s** (en respectant les règles décrites précédemment 😊).

OBJECTIF N°4 : Création d'une API REST

Exposez les données nettoyées via une API REST Spring Boot, incluant les endpoints suivants :

- **GET /products/top-by-brand?brand=X&limit=N : top N produits pour une marque**
- **GET /products/top-by-category?category=X&limit=N : top N produits pour une catégorie**
- **GET /products/top-by-brand-category?brand=X&category=Y&limit=N**
- **GET /ingredients/top?limit=N : N ingrédients les plus courants**
- **GET /allergens/top?limit=N**
- **GET /additives/top?limit=N**

Instructions techniques complémentaires

- Créer des groupes de travail de 2 (voire 3) personnes au maximum
- Utiliser IntelliJ ou votre IDE préféré
- Créer un nouveau projet Spring Boot (Maven) nommé « **etl-off** »
- Créer le répertoire /conception à la racine du projet (y placer les éléments de conception)
- Documenter le code, commit régulièrement
- Utiliser votre plateforme de collaboration préférée pour versionner le projet et me le partager ([@ssy-sdv](#))
- Mettre en place les optimisations afin de battre le record !

Bonus possible

- Ajouter de l'observabilité : Actuator + Prometheus
- Exposer des métriques de performance
- Tester la scalabilité avec un grand volume de données